

ADVISOR: Approximate Computing-frienDly High-LeVel Synthesis DesIgn Space ExplORer

B. Parchamdar and B. Carrion Schaefer

The University of Texas at Dallas

Department of Electrical and Computer Engineering

{baharealsadat.parchamdar,schaferb}@utdallas.edu

Abstract—Approximate computing is a relatively new computing paradigm that allows to trade-off area/power with accuracy. Another relatively new VLSI design trend is to raise the level of design abstraction from the Register-Transfer Level (RTL) to the behavioral level and use High-Level Synthesis (HLS) to synthesize these behavioral descriptions. HLS has one unique advantage over RTL design. It completely decouples the functional description from the implementation details. This allows to design and verify the behavioral description once, but then generate a large number of functional equivalent hardware implementations of unique area vs. performance trade-offs. This is typically achieved through synthesis directives in the form of pragmas that the HLS user annotates at the source code to mainly control how to synthesize arrays (RAM, registers), loops (unroll, pipeline) and functions (inline or not).

In this work we leverage this uniqueness and build an automated HLS design space explorer to find the hardware circuit most amenable to approximate computing, this is, has the highest potential for area/power savings. We have coined this explorer ADVISOR. The main problem with traditional exploration approaches is their long run time, which is worse in this case because every new implementation needs to be approximated to fully understand the trade-offs in terms of area/power vs. error. Thus, in order to accelerate this exploration process, we propose to evaluate each new designs based on an Approximation Friendliness Index (AFI) that can be computed statically, very fast, and only fully approximate the designs recommended by our flow that have high AFI values. Experimental results show that this approach leads to basically the same results as exhaustively approximating every new design, while being on average $68\times$ faster.

Index Terms—High-Level Synthesis, Design Space Exploration, Approximate Computing.

I. INTRODUCTION

Creating faster, smaller (cheaper) and lower-power integrated circuits (ICs) is paramount to continue supplying hardware devices that can accomplish more computationally intensive tasks while extending their battery life at smaller costs. This requires new VLSI design methodologies and computing paradigms that either complement or substitute traditional design and optimization techniques. Two of these methodologies that we address in this work are approximate computing and High-Level Synthesis (HLS).

Approximate computing leverages the fact that many applications tolerate errors at their output. These applications include Digital Signal Processing (DSP) and image processing applications. This can be leveraged to build smaller and/or lower power hardware implementations. This has become ever more relevant these days where modern ICs now include multiple hardware accelerators that specifically deal with these types of applications [1], [2].

At the same time HLS is often used to build these hardware accelerators which are typically first designed and verified using high-level languages like C, C++ or Matlab and then converted into efficient hardware implementations. HLS allows to directly synthesize these high-level descriptions into hardware (RTL). HLS has been shown to have many advantages over low-level Hardware Description Languages (HDLs) like reducing the turn-around time, allowing faster simulations and the ability to generate different types of hardware circuits from the same behavioral descriptions by simply setting different synthesis options [3].

This last advantage will be leveraged in this work to find an RTL implementation (micro-architecture) of a hardware accelerator that is amenable to approximate computing that has the highest potential for approximation. In particular we show that some implementations cannot be approximate at all (e.g., because of the high degree of resource sharing), while other implementations are very amenable to approximations. We call our proposed flow ADVISOR: Approximate Computing-frienDly High-LeVel Synthesis DesIgn Space ExplORer. In summary, the main contributions of this work are:

- Introduce an automated HLS design space explorer that has the goal of finding the most approximation friendly hardware implementations.
- Introduce an approximation friendliness index that can be quickly computed for RTL descriptions that significantly accelerates the design space exploration process.
- Present a comprehensive evaluation showing the effectiveness of our proposed flow.

II. RELATED WORK

Many classes of applications exhibit significant tolerance to inaccuracies in their computations. Some examples include image processing, multimedia applications, and machine learning [1], [2], [4]. These inaccuracies can be exploited to build circuits with smaller areas, lower power, and higher performance. **Table I** highlights some of the most salient work, categorizing them based on: (1) The VLSI design stage where they are applied, e.g., HLS [5]–[10], logic synthesis [11]–[13] and physical design [14] or across all design stages [15]. (2) Specific approximate arithmetic unit (e.g., adders and multipliers), (3) approximate architectures like reconfigurable fabrics and approximate interconnects [16], [4], and (4) system-level optimizations [17], [18]. The main problem with these previously developed techniques is that they do not scale well and for larger circuits the runtime is extremely large without even the guarantee that the circuit can be approximated.

TABLE I: Overview of the state of the art in approximate computing classified by VLSI design stage and component.

High-Level Synthesis: Joint precision [5], ABACUS [6], HLS schedule [7] AHLS [8], HLS real time [9], AxHLS [10]
Logic Synthesis: SALSA [11], GALS [12], Boolean network [13]
Physical Design: FPGA wire removal [14]
Multi VLSI stage approximation: HLS to gate netlist [15]
Functional Units: Adders: [19], [20], Multipliers: [21]–[23]
Interconnect: AxBA [4], [16]
System-Level/Cross-Layer approximations: SoC: [17], [18]

To address this, the authors in [24] proposed a fast design space exploration for RTL descriptions to find quickly find Pareto-optimal configurations by intelligently assigning arithmetic functional to approximate versions. This previous work takes a given Verilog descriptions as input and hence the results are highly dependent on how amenable this architecture is to approximation.

Approximate computing can also be classified based on the type of approximation. Some approximation can be controlled dynamically at runtime like dynamic voltage overscaling where the supply voltage is reduced beyond V_{dd} [25], [26] and static approximations where the result of the approximation leads to a fixed hardware circuit (most common). Our work deals this latter type of approximations. To the best of our knowledge this is the first work that aims at leveraging the untimed behavioral description to quickly find the RTL descriptions that have the highest potential for approximation.

III. BACKGROUND

This section covers important background information required to better understand the proposed contribution. In particular it will cover the basics of HLS and showcase some approximate computing primitives.

High-Level Synthesis: HLS converts untimed behavioral descriptions like C or C++ into RTL (Verilog or VHDL).

In order to allow users to generate a specific hardware architecture, HLS vendors make extensive use of synthesis directives in the form of pragmas to mainly control how to synthesize arrays, loops and functions. Arrays can be synthesized as registers or RAM, loops unrolled, partially unrolled, not unrolled or pipelined and functions inlined or not. Setting different combinations of these pragmas lead to hardware implementations of different area vs. performance trade-offs. In the code snippet in Fig. 1 if the array is synthesized as registers and the loop fully unrolled, then a hardware circuit of low latency but high area is obtained. In contrast, setting the pragma combinations to not unroll the loop and still synthesizing the array as registers leads to the smallest, but slowest implementation. The back end of the HLS process takes these results and outputs the synthesizable RTL.

As shown in Fig. 1, setting different pragma combinations leads to HW implementations of unique area vs. performance trade-offs. Because area and performance are conflicting design metrics there is no single HW implementation that dominates all of the others. Instead, out of all the pragma combinations there are some that lead to Pareto-optimal designs that form the Pareto-front ($\bar{P}$). The process of finding

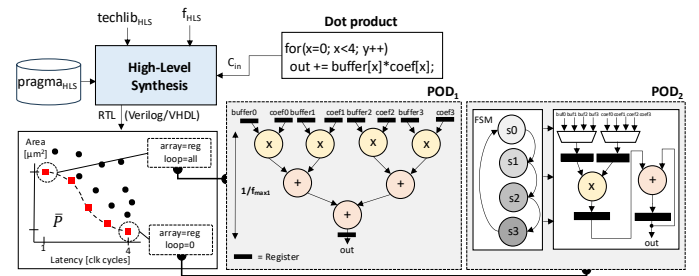


Fig. 1: Motivational example of dot product synthesized using different HLS pragma combinations. Pareto-optimal design (POD_1): Exposes 6 internal signals and 7 functional units (FUs). Pareto-optimal design (POD_2): Exposes 4 internal signal and 2 FUs.

these Pareto-optimal designs can be automated and is typically referred to as HLS design space exploration (DSE) [27]. Here, our goal is not to find the Pareto-optimal designs, but the design that is most amenable to approximate computing within a given area, performance and power constraints.

Approximate Computing: Approximate computing is based around a variety of different primitives that can be applied to different individual VLSI design stages like HLS [5], [6], [8], logic synthesis [11], [12] and physical design [14]. Some approximations are specific to a component in the SoC like on-chip interconnect [4] or arithmetic operations. These include approximated adders [19], [20], [24] and multipliers [21], [22] which have been widely studied due to their importance.

Fig. 2 continues the example shown in Fig. 1 where the design with highest performance (lowest latency) circuit implementation is further approximated. In this case we can see that the for-loop is fully unrolled in order to expose the four multiplications and three additions. Two different approximations are shown here. In $POD_{approx1}$, the exact functional units (FUs) (adders and multipliers) are substituted by approximated versions. Based on the type of approximate FUs and the input data, different substitutions can lead to different area/power savings while leading to different output errors. In $POD_{approx2}$ we can observe other approximation primitives called Variable to Variable (V2V) or Variable to Constant (V2C) substitution. In V2C, If any signal in the circuit *hovers* around a certain mean value with small standard deviation, then, this signal can be directly connected to a constant of that mean value. Additionally, if two signals are highly correlated one signal can be connected to another signal, directly or after re-sizing it. This is a V2V approximation.

As shown in Fig. 2 typically V2V and V2C approximations have a higher impact on the area/power reduction because they can eliminate entire portions of the circuit. As shown in the approximated circuits in Fig. 2, $POD_{approx2}$, has only two multipliers and three adders and hence, is much smaller than $POD_{approx1}$. This observation will be leveraged in this work to determine the order in which we approximate a design.

In the case of the smallest POD in Fig. 1 (POD_2), no approximation was possible (FU nor V2V or V2C) because the FSM cannot be approximated and the FUs execute different operations from the application (resource sharing). This

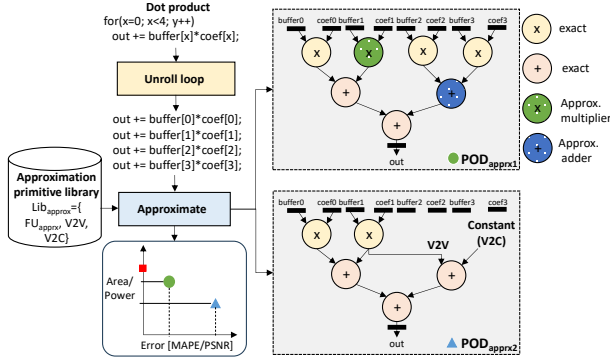


Fig. 2: Overview of approximate computing example including two different approximation primitives: Functional Unit approximation (FU_{approx}) and Variable to Variable and Variable (V2V) to Constant (V2C) approximations.

implies that they execute a larger variety of different data distributions, and hence, it is not possible to approximate these. Based on this, and without any loss of generality, we can make the following observations:

Observation 1: Designs with high amount of resource sharing are not good candidates for approximation. These designs include large FSMs which cannot be approximated and shared FUs which execute different operations with different data distributions on them.

Observation 2: Designs with larger amounts of parallelism contain a large number of FUs that only execute one operation and also have many internal signal have higher chances of being successfully approximated.

Observation 3: V2V and V2C approximations typically lead to larger area/power reductions as large portions of the circuits can be completely removed. Also, the closer the approximation is to the output the higher the savings are as larger parts of the circuit can be pruned.

IV. PROPOSED FLOW

Fig. 3 shows an overview of the complete flow. The inputs to the proposed flow are the synthesizable behavioral description (C_{HLS}) for HLS, a library with synthesis directives for HLS to mainly control how to synthesize arrays, loops and functions ($pragma_{lib}$), the HLS technology library ($techlib_{HLS}$) and target synthesis frequency (f_{HLS}). In addition, a library of approximation primitives. In this work use V2V, V2C, and FU approximations, but other approximations like voltage over-scaling could also be included. Finally, the maximum allowable output error that this application tolerate (E_{max}) and the test vectors that represent the workload at which this design will be subject to (TV) also need to be given. Based on the type of application the E_{max} could be given as Mean Average Percentage Error (MAPE) or PSNR (for image processing applications).

The proposed flow is composed of two phases as shown in the figure. Phase I performs the approximation friendly HLS DSE and is further composed of three steps. Step 1 generates a new set of HLS synthesis directives, step 2 synthesizes the behavioral description with these new pragmas, and step

3 computes our newly proposed approximation friendliness index (AFI). These steps are repeated until a given exit condition is met (e.g., the AFI index is maximized). Phase II then approximates the design with highest AFI. The next subsections describe these phases in detail.

Phase I: Approximation friendly HLS Design Space Exploration

This first phase is the main contribution of this work. The idea is to quickly find an RTL implementation that has the maximum potential for approximation. As shown in the previous sections, some optimizations like resource sharing do not allow to approximate the circuit. Hence, this phase needs to filter out those design implementations and return other implementations with high potential for approximation. The alternative to this approach would be to generate a new RTL description for every new pragma combination, fully approximate it, and then evaluate its resultant area/power vs. output error. This is not only time consuming, but also not feasible for behavioral descriptions that have a larger number of unique pragma combinations. Hence, faster and more efficient approaches are needed.

Step 1: New Pragma Combination Generation: This first step starts by generating a random new pragma combination for the arrays, loops and functions in the given behavioral description (C_{HLS}). Although initially this pragma combination is random this phase updates a small portion of these pragmas through *fuzzing* in each iteration. Fuzzing has been widely used in verification and testing [28]–[30] where test vectors are modified (mutated) to detect hard to find bugs in hardware circuits. In the context of testing and verification the idea is to *fuzz* (update) the test vectors in order to find bugs and corner cases in the design. New test vectors are generated from one iteration to another by mutating (updating) a small number of test vectors between iterations. In the context of HLS DSE a pragma combination is equivalent to the test vectors and in every new iteration our flow mutates a small portion of the pragmas. Empirically we tried different mutation rates and found that updating 10% of the pragmas each iteration led to good results (quality of results and runtime). We used the following cost function to drive this process:

$$\begin{aligned} & \text{maximize} \quad \text{Approximation Friendly Index (AFI)} \\ & \text{subject to:} \quad A_{max}, L_{max} \end{aligned} \quad (1)$$

where the Approximation Friendly Index (AFI) estimates how amenable the circuit is to the approximations given in the approximation library (Lib_{approx}), and A_{max} and L_{max} are optional and represent the maximum area and latency that the final circuit should meet. The goal is to find the circuit that meets the given constraints with the highest possible AFI. The output of this step is the new behavioral description with the updated pragma combination (C_{pragma}).

Step 2: High-Level Synthesis: This next step takes as input the newly generated pragma combination (C_{pragma}) and synthesizes (HLS) if. The following inputs are needed by the HLS tool: C_{pragma} , $techlib_{HLS}$ and f_{HLS} . The results are the newly generated RTL code (RTL_{new}) and a report file

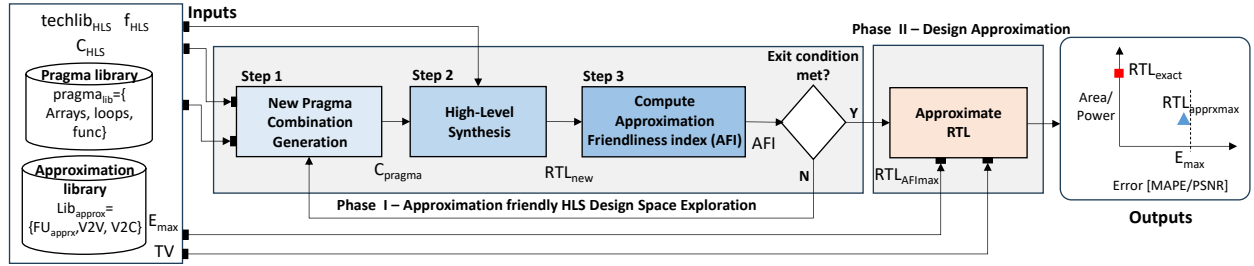


Fig. 3: Overview of proposed flow composed of 2 phases steps: **Phase I:** Approximation friendly HLS design space exploration composed of three step: **Step 1:** Generate synthesis directives for HLS. **Step 2:** Synthesize design (HLS). **Step 3:** Compute proposed approximation friendliness index (AFI). **Phase II:** Approximate RTL code.

that summarizes the area and performance of this hardware implementation.

Step 3: Compute Approximation Friendliness Index (AFI):

This last step parses the newly generated RTL code (RTL_{new}) and computes the *Approximation Friendliness Index* (AFI). The AFI should have two main characteristics: (1) It should be easy and fast to compute and (2) it should correlate well with the potential of the RTL to be approximated.

Based on this The AFI is computed by statically parsing the RTL description and is highly dependent on the approximation primitives used (given as input to the flow). Table II highlights three cases based on three different approximation primitives. For the first case (V2V and V2C), we use internal signals. The intuition is that the more internal signals the circuit has, the higher the probability is that a V2V or V2C approximation is found. Larger number of internal signals also serves as a proxy for larger amount of parallelism and hence less resource sharing, which hampers approximations. Hence, for this first case our method parses the RTL code and counts the internal signals in the RTL as AFI. It should be noted that HLS generates designs that contains a FSM (controller) and a data path. Because the FSM cannot be approximated, we only consider the signals in the data path.

In the case of other approximation primitives like FUs approximations, the AFI simply counts the number of FUs in the newly generated RTL code. The more FUs, the higher the possibility of *finding* an approximation. Lastly when considering other approximations like voltage over-scaling, where the supply voltage (V_{dd}) is reduced beyond the nominal voltage, the AFI is proportional to the positive timing slack of the circuit, as voltage over-scaling reduces the switching speed of the transistors and hence, having a larger positive timing slack allows for higher voltage reductions. Table II shows an example of how to compute the AFI for the two implementations of the dot-product shown in Figure 1. POD_1 has a higher AFI index for the first two approximations, while smaller AFI in the timing slack case as compared to POD_2

TABLE II: AFI predictor based on approximation primitive

AFI Predictor	Approximation	AFI POD_1/POD_2^*
$\sum$ Signals in datapath	V2V, V2C	7/3
$\sum$ Functional units (FUs)	FUs	7/2
Positive timing slack	Voltage over-scaling	2.3/6.2 [†]

*AFI computation example from Pareto-optimal examples from Figure 1

[†]Nangate 40nm technology and HLS frequency 100Mhz

as this last design does not chaining the FUs and hence, the register to register delay is smaller.

In the case that of multiple approximation primitives, then the different predictive models are combined to form a single AFI predictor and weights on each predictor set to determine which approximation should have higher preference. This phase ends by sorting the explored designs based on AFI.

Phase II: Approximate RTL

This second phase takes as inputs the list of explored designs sorted by AFI ($RTL_{AFI_{max}}$), the maximum allowable error threshold (E_{max}) and the test vectors (TV) that represent the typical workload of this circuit. It also requires the logic synthesis technology library ($techlib_{LS}$) in order to reveal the effect of the approximations on the final synthesized circuit. The output is the approximated RTL design that leads to the largest area savings ($RTL_{approxmax}$) while leading to an error that is within the specified maximum error threshold (E_{max}). In this work we consider two approximation primitives: V2V/V2C and FUs approximations. Based on the discussion in section III and as shown in Fig. 2 some approximations typically lead to larger savings than others. E.g., a V2V approximation can remove entire parts of a circuit including FUs, while FUs approximations keep the original circuit, and only affect locally the approximated FU. Based on this our flow starts by executing the most aggressive approximations first (in this case V2V and V2C) and then *fine-tune* the process by approximating the remaining FUs.

Because in this work we only target two approximations, this phase is composed of 2 steps shown in Fig. 4. The next subsections describe these in detail:

Step 1: V2V and V2C Approximations: This phase takes as input the design found in phase I with the highest AFI ($RTL_{AFI_{max}}$) and tries to find any V2V and V2C approximations such that the circuit can be simplified. For this, it starts by simulating it with the given TV. A VCD file dumping the simulation result including all of the internal signals is generated after the simulation. This generated VCD file is parsed and the mean (μ) and standard deviation (σ) computed for every internal signal in $RTL_{AFI_{max}}$. We also compute the correlation between all the internal signals.

Next, our flow analysis the results and extract all possible approximation points. These are signals in $RTL_{AFI_{max}}$ that have the potential of being approximated through V2V or V2C

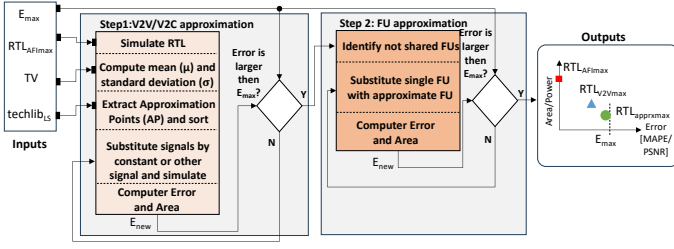


Fig. 4: Phase II Design Approximation overview of proposed flow composed of 2 phases.

approximations. For this we consider all of the signals with a standard deviation (σ) equal to E_{max} and correlation factor less than E_{max} %. At this stage we only select the signals that have the potential to approximate the circuit.

Our flow also traverses $RTL_{AFI_{max}}$ and sorts the approximations based on closeness to the primary outputs. Substituting a signal close to the outputs should intuitively lead to larger savings as a larger portion of the circuit can be pruned as opposed to approximating a signal closer to the primary inputs. Based on this, the flow proceeds to sort all of the approximation points (AP_{list}) based on their *closeness* the primary outputs. Our flow continues by setting the different approximations from AP_{list} following a greedy algorithm. Basically, the approximation closest to the primary outputs is enabled first (either V2V or V2C). A new RTL description is generated with this approximation (RTL_{approx}). This RTL description is simulated, and the output error computed. This process is repeated by enabling the next closest approximation to the outputs in RTL_{approx} while the measured error does not exceed E_{max} . If the measured error exceeds E_{max} , then the last approximation is removed and previous iteration RTL_{approx} finally synthesized with the given logic synthesis technology library ($techlib_{LS}$). This will enable us to full characterize the area savings vs. output error. Note that our flow does not need to remove the unused logic as the logic synthesizer will automatically do this as it will eliminate any circuit part that does not affect the outputs. It should also be noted that if a V2V or V2C was successful then some of the approximation points extracted might not exist anymore as part of the circuit is removed during the approximation. These approximation points are therefore removed.

In order to compute the error, different error metrics have been proposed in the past [31], [32]. In this work we use the Mean Absolute Percentage Error (*MAPE*) for signal processing applications and Peak Signal to Noise Ratio (*PSNR*) for image processing applications, as PSNR is often used as a measure of quality in images [33]. *MAPE* calculates the error between the output of the approximate circuit and the golden output from the exact solution as follows: $MAPE = \frac{1}{N} \sum_{i=1}^N \left| \frac{GO_i - APO_i}{GO_i} \right|$, where GO_i is the error-free output (golden output), and APO_i is the approximate output. PSNR on the other hand is computed here: $PSNR = 10 \log_{10} \frac{MAX^2}{MSE}$, where MSE is the Mean Squared Error from the noise-free image and MAX the maximum pixel value of the original image.

The maximum allowable error is domain specific and hence, requires domain expertise. In our case we set the maximum error (E_{max}) to typical values reported in approximate computing literature [1], [2], [4] and study the robustness of our approach when the E_{max} changes. These include MAPE=10%, PSNR=20db and MAPE=20%, PSNR 10db.

The result of this step is the approximated RTL description RTL_{v2vmax} .

Step 2: FU Approximation: This next step takes as input the approximated RTL code and continues the approximation process by approximating only the FUs.

For this, this step starts by parsing RTL_{v2vmax} and extracting the FUs that are not shared. This is done by checking if the FUs' input is connected to muxes or not. As mentioned before, FU sharing does not allow FU approximation as multiple operations are mapped onto the same FU. Once these FUs are identified, our method starts by substituting iteratively following a greedy algorithm the largest FUs in the list. For this we use EvoApproxLib [34] an open-source library of approximated adders and multipliers.

These two step can be executed for only the RTL design with highest AFI or it can be repeated for the N designs with highest AFI. In the experimental results section we execute both scenarios where in the case of repeating this phase we do this until no smaller design withing E_{max} can be found.

The final output of the smallest design, $RTL_{approxmax}$, with an error within E_{max} .

TABLE III: Benchmark characteristics

	Bench	Lines	Mul	Add/Sub	Input Stimuli	Error
DSP	fir	90	8	8	10,000 Random	MAPE
	fft	193	16	88	10,000 Random	MAPE
	interp	207	24	24	10,000 Random	MAPE
	decim	220	30	81	10,000 Random	MAPE
Image	sobel	70	0	45	512x512 BMP	PSNR
	idct	362	103	164	512x512 BMP	PSNR

V. EXPERIMENTAL RESULTS

Six computationally intensive applications amenable to approximate computing were used to test our proposed method. These designs were taken from the open source Synthesizable SystemC Benchmark suite (S2CBench) [35]. Table III shows all the applications and highlights their main characteristics in terms of the number of lines of code, functional units required when the loops are fully unrolled (default HLS mode). The benchmarks can be coarsely classified into digital signal processing (fir, fft, decimation and interpolation filter) and image processing (sobel, idct). These families of designs have shown to be inherently error resilient.

Table IV summarizes the experimental setup in terms of the HLS and LS tool used and the server where we ran the experiments.

Three approaches were compared in this work. In the first one we use the same fuzzing-based design space exploration, but fully approximate each design generated instead of relying on the AFI index to determine which design to approximate. The cost function used in this case is not the

TABLE IV: Experimental Setup

HLS Tool	NEC CyberWorkBench 10.1 [36]
Logic Synthesis	Synopsys Design Compiler 2018.06-SP1
RTL Simulator	Synopsys VCS 2018.06
Synthesis Technology	Nangate 45nm Open Cell Library
Frequency (HLS and LS)	100 MHz
CPU	Intel Core i7-4790 CPU @3.6GHZ
Memory	16 GB DDR3
Operating System	CentOS v8.0

AFI index, but the actual area savings obtained after the approximation flow (phase 2) is fully executed for each design. This approach represents the naïve approach (**NAIVE**). In the second and third approach we use our proposed approach. In the first case we only approximate the design with highest AFI (**ADVISOR(AFI=1)**), while in the second approach we approximate the N designs with highest AFI (**ADVISOR(AFI=N)**) until a design with smallest area cannot be found. In all cases, phase I exits if after 10 consecutively generated designs, the area savings or AFI cannot be improved for the **NAIVE** and **ADVISOR** approach respectively.

Fig. 5 and Table V summarize the area savings and speedup obtained by the different approaches for different maximum error thresholds (E_{max}). Figs. 5(a) show the results in the unconstraint case, when no area (A_{max}) and latency (L_{max}) constraint are set and Figs. 5(b) when a constraint is set to half of the area of the largest design and half the latency of the slowest design. This will help us to fully characterize our approach under different scenarios. From the results we can make the following observations:

Observation 1: All of the benchmarks could be successfully approximated. For the unconstraint case, on average the area reduction was 48% and 52% for the **ADVISOR(AFI=1)** and **ADVISOR(AFI=N)** and MAPE=10%, PSNR=20db case respectively. The area reduction was approximate 10% higher to 59% and 66% when E_{max} was further relaxed to MAPE=20%, PSNR=10db. Moreover, The results obtained by our approach are very similar to the **NAIVE** approach, in particular in the **ADVISOR(AFI=N)** case.

Observation 2: Approximating only the design with highest AFI index leads to very good results, but on average 2% and 3% worse then continuing to approximate the N designs with highest AFI. On average, the five designs with highest AFI were approximated ($N=5$).

Observation 3: The unconstraint exploration case leads to much better results then the constraint case. On average the area reduction worsen between 31 to 37% for the different E_{max} and **ADVISOR** approaches. This is mainly due to the fact that limiting the search space to generate a smaller circuits forces the explorer to discard circuits with small or no resource sharing. We noticed that in for this case very few V2V and V2C approximations were found and that almost all of the area reduction is due to FU approximations.

Observation 4: From Table V we can observe that our flow is on average $66\times$ and $25\times$ faster than the **NAIVE** approach for **ADVISOR(AFI=1)** and **ADVISOR(AFI=N)** respectively,

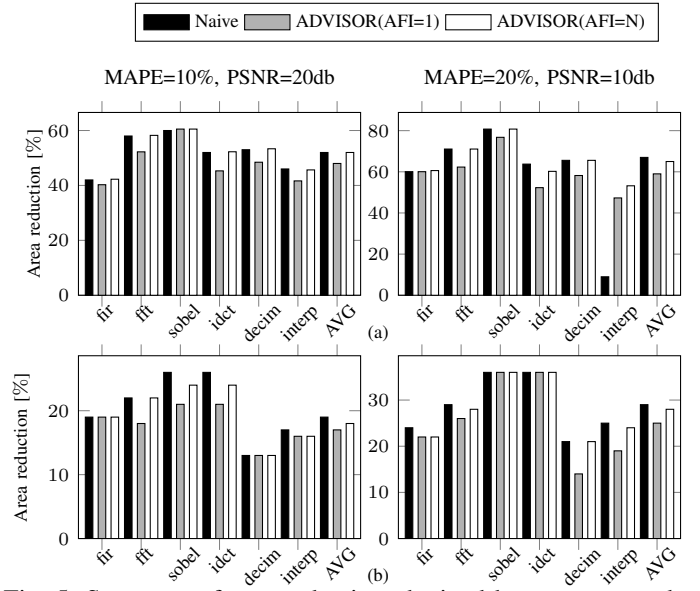


Fig. 5: Summary of area reduction obtained by our proposed flow (**ADVISOR**) vs. Naive search (**NAIVE**); (a) when no Area (A_{max}) and Latency (L_{max}) constraints are set; (b) when maximum area and latency constraint are set to half the max Area and Latency for different E_{max} .

while leading to similar results. This is one of the main contributions of this work, and we expect even higher speedups as more approximation primitives are added to the process.

TABLE V: Runtime comparison of NAIVE vs. proposed flow

Benchmark	Runtime [h:mm]		
	NAIVE	ADVISOR (AFI=1)	ADVISOR (AFI=N)
fir	7:45	0:06	0:14
fft	13:07	0:12	0:33
decimation	23:12	0:25	0:59
interpolation	26:56	0:37	1:33
sobel	19:23	0:13	0:43
idct	21:12	0:15	0:45
Geomean	16:53	0:15	0:41

Based on these results we believe that the proposed fully automated flow is very efficient finding the most amenable RTL design to approximate computing given an untimed behavioral description for HLS.

VI. CONCLUSION

In this work, we have introduced an automated High-Level Synthesis design space explorer that has the goal of finding the RTL implementation with the highest potential for approximation. Previous work on RTL approximation assumes that the RTL is given beforehand. Fortunately, HLS has the ability to generate a larger number of functional equivalent circuits with different hardware implementations (RTLs). We leverage this fact and propose a very simply approximation friendly index to find these designs. Future work includes adding more approximation primitives.

ACKNOWLEDGMENT

This work is partially supported by the NSF through award number #2323820.

REFERENCES

- [1] Q. Xu, T. Mytkowicz, and N. S. Kim, "Approximate computing: A survey," *IEEE Design Test*, vol. 33, no. 1, pp. 8–22, Feb 2016.
- [2] S. Mittal, "A survey of techniques for approximate computing," *ACM Comput. Surv.*, vol. 48, no. 4, pp. 62:1–62:33, Mar. 2016.
- [3] B. Carrion Schaefer, *High-Level Synthesis Made Easy*, 1st ed. highX Technologies, 2023. [Online]. Available: <https://www.hlsbook.com>
- [4] F. Betzel *et al.*, "Approximate communication: Techniques for reducing communication bottlenecks in large-scale parallel systems," *ACM Comput. Surv.*, vol. 51, no. 1, Jan. 2018.
- [5] C. Li, W. Luo, S. S. Sapatnekar, and J. Hu, "Joint precision optimization and high level synthesis for approximate computing," in *DAC*, June 2015, pp. 1–6.
- [6] K. Nepal, Y. Li, R. I. Bahar, and S. Reda, "ABACUS: A technique for automated behavioral synthesis of approximate computing circuits," in *DATE*, March 2014, pp. 1–6.
- [7] S. Xu and B. Carrion Schaefer, "On the design of high performance hw accelerator through high-level synthesis scheduling approximations," in *DATE*, 2020, pp. 1378–1383.
- [8] S. Lee, L. K. John, and A. Gerstlauer, "High-level synthesis of approximate hardware under joint precision and voltage scaling," in *Design, Automation Test in Europe Conference Exhibition (DATE)*, 2017, 2017, pp. 187–192.
- [9] M. T. Leipnitz and G. L. Nazar, "High-level synthesis of approximate designs under real-time constraints," *ACM Trans. Embed. Comput. Syst.*, vol. 18, no. 5s, Oct. 2019. [Online]. Available: <https://doi.org/10.1145/3358182>
- [10] J. Castro-Godínez, J. Mateus-Vargas, M. Shafique, and J. Henkel, "Axlhs: Design space exploration and high-level synthesis of approximate accelerators using approximate functional units and analytical models," in *2020 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*, 2020, pp. 1–9.
- [11] S. Venkataramani, A. Sabne, V. Kozhikkottu, K. Roy, and A. Raghunathan, "Salsa: Systematic logic synthesis of approximate circuits," in *DAC*, 2012, pp. 796–801.
- [12] J. Miao, A. Gerstlauer, and M. Orshansky, "Approximate logic synthesis under general error magnitude and frequency constraints," in *ICCAD*, 2013, pp. 779–786.
- [13] Y. Wu and W. Qian, "An efficient method for multi-level approximate logic synthesis under error rate constraint," in *DAC*, 2016, pp. 1–6.
- [14] Y. Wu, C. Shen, Y. Jia, and W. Qian, "Approximate logic synthesis for fpga by wire removal and local function change," in *ASP-DAC*, Jan 2017, pp. 163–169.
- [15] S. Xu and B. Carrion Schaefer, "Exposing approximate computing optimizations at different levels: From behavioral to gate-level," *IEEE TVLSI*, vol. 25, no. 11, pp. 3077–3088, 2017.
- [16] J. R. Stevens, A. Ranjan, and A. Raghunathan, "Axba: An approximate bus architecture framework," in *ICCAD*, 2018, pp. 1–8.
- [17] A. Agrawal, J. Choi, K. Gopalakrishnan, S. Gupta, R. Nair, J. Oh, D. A. Prener, S. Shukla, V. Srinivasan, and Z. Sura, "Approximate computing: Challenges and opportunities," in *2016 IEEE International Conference on Rebooting Computing (ICRC)*, 2016, pp. 1–8.
- [18] M. Shafique, R. Hafiz, S. Rehman, W. El-Harouni, and J. Henkel, "Invited: Cross-layer approximate computing: From logic to architectures," in *DAC*, 2016, pp. 1–6.
- [19] V. Gupta, D. Mohapatra, A. Raghunathan, and K. Roy, "Low-power digital signal processing using approximate adders," *IEEE TCAD*, vol. 32, no. 1, pp. 124–137, 2013.
- [20] M. Shafique, W. Ahmad, R. Hafiz, and J. Henkel, "A low latency generic accuracy configurable adder," in *DAC*, 2015, pp. 1–6.
- [21] C. Liu, J. Han, and F. Lombardi, "A low-power, high-performance approximate multiplier with configurable partial error recovery," in *2014 Design, Automation Test in Europe Conference Exhibition (DATE)*, 2014, pp. 1–4.
- [22] H. Jiang, C. Liu, N. Maheshwari, F. Lombardi, and J. Han, "A comparative evaluation of approximate multipliers," in *2016 IEEE/ACM International Symposium on Nanoscale Architectures (NANOARCH)*, 2016, pp. 191–196.
- [23] Y. Guo, Q. Zhou, and H. Sun, "High-efficiency fpga-based approximate multipliers with lut sharing and carry switching," in *2024 DATE*, 2024, pp. 1–6.
- [24] V. Mrazek, M. A. Hanif, Z. Vasicek, L. Sekanina, and M. Shafique, "autoax: An automatic design space exploration and circuit building methodology utilizing libraries of approximate components," in *DAC*, 2019, pp. 1–6.
- [25] J. Zhang, K. Rangineni, Z. Ghodsi, and S. Garg, "Thundervolt: Enabling aggressive voltage underscaling and timing error resilience for energy efficient deep learning accelerators," in *2018 55th ACM/ESDA/IEEE Design Automation Conference (DAC)*, 2018, pp. 1–6.
- [26] D. Mohapatra, "Approximate computing: Enabling voltage over-scaling in multimedia applications," Ph.D. dissertation, Purdue University, 2011.
- [27] B. C. Schaefer and Z. Wang, "High-level synthesis design space exploration: Past, present, and future," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 39, no. 10, pp. 2628–2639, 2020.
- [28] R. Dutra, R. Gopinath, and A. Zeller, "Formatfuzzer: Effective fuzzing of binary file formats," vol. 33, no. 2, 2023.
- [29] C. Zhang and J. Chen, "Fuzzing methods recommendation based on feature vectors," in *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2021, pp. 1079–1081.
- [30] R. Saravanan and S. M. Pudukotai Dinakarrao, "The fuzz odyssey: A survey on hardware fuzzing frameworks for hardware design verification," in *Proceedings of the Great Lakes Symposium on VLSI 2024*, ser. GLSVLSI '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 192–197. [Online]. Available: <https://doi.org/10.1145/3649476.3658697>
- [31] D. S. Khudia, B. Zamirai, M. Samadi, and S. Mahlke, "Quality control for approximate accelerators by error prediction," *IEEE Design Test*, vol. 33, no. 1, pp. 43–50, Feb 2016.
- [32] J. Liang, J. Han, and F. Lombardi, "New metrics for the reliability of approximate and probabilistic adders," *IEEE Transactions on Computers*, vol. 62, no. 9, pp. 1760–1771, Sept 2013.
- [33] R. Dosselmann and X. D. Yang, "Existing and emerging image quality metrics," in *Canadian Conference on Electrical and Computer Engineering*, May 2005, pp. 1906–1913.
- [34] V. Mrazek, R. Hrbacek, Z. Vasicek, and L. Sekanina, "Evoapprox8b: Library of approximate adders and multipliers for circuit design and benchmarking of approximation methods," in *Design, Automation Test in Europe Conference Exhibition (DATE)*, 2017, 2017.
- [35] B. Carrion Schaefer and A. Mahapatra, "S2CBench: Synthesizable SystemC Benchmark Suite," *IEEE Embedded Systems Letters*, vol. 6, no. 3, pp. 53–56, 2014.
- [36] NEC CyberWorkBench, 2019. [Online]. Available: www.cyberworkbench.com